
Algorithm 1 Minibatch version of the Auto-Encoding VB (AEVB) algorithm. Either of the two SGVB estimators in section 2.3 can be used. We use settings $M = 100$ and $L = 1$ in experiments.

```

 $\theta, \phi \leftarrow$  Initialize parameters
repeat
   $\mathbf{X}^M \leftarrow$  Random minibatch of  $M$  datapoints (drawn from full dataset)
   $\epsilon \leftarrow$  Random samples from noise distribution  $p(\epsilon)$ 
   $\mathbf{g} \leftarrow \nabla_{\theta, \phi} \tilde{\mathcal{L}}^M(\theta, \phi; \mathbf{X}^M, \epsilon)$  (Gradients of minibatch estimator (8))
   $\theta, \phi \leftarrow$  Update parameters using gradients  $\mathbf{g}$  (e.g. SGD or Adagrad [DHS10])
until convergence of parameters  $(\theta, \phi)$ 
return  $\theta, \phi$ 

```

Often, the KL-divergence $D_{KL}(q_\phi(\mathbf{z}|\mathbf{x}^{(i)})||p_\theta(\mathbf{z}))$ of eq. (3) can be integrated analytically (see appendix B), such that only the expected reconstruction error $\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x}^{(i)})} [\log p_\theta(\mathbf{x}^{(i)}|\mathbf{z})]$ requires estimation by sampling. The KL-divergence term can then be interpreted as regularizing ϕ , encouraging the approximate posterior to be close to the prior $p_\theta(\mathbf{z})$. This yields a second version of the SGVB estimator $\tilde{\mathcal{L}}^B(\theta, \phi; \mathbf{x}^{(i)}) \simeq \mathcal{L}(\theta, \phi; \mathbf{x}^{(i)})$, corresponding to eq. (3), which typically has less variance than the generic estimator:

$$\tilde{\mathcal{L}}^B(\theta, \phi; \mathbf{x}^{(i)}) = -D_{KL}(q_\phi(\mathbf{z}|\mathbf{x}^{(i)})||p_\theta(\mathbf{z})) + \frac{1}{L} \sum_{l=1}^L (\log p_\theta(\mathbf{x}^{(i)}|\mathbf{z}^{(i,l)}))$$

where $\mathbf{z}^{(i,l)} = g_\phi(\epsilon^{(i,l)}, \mathbf{x}^{(i)})$ and $\epsilon^{(i,l)} \sim p(\epsilon)$ (7)

Given multiple datapoints from a dataset $\mathbf{X}$ with N datapoints, we can construct an estimator of the marginal likelihood lower bound of the full dataset, based on minibatches:

$$\mathcal{L}(\theta, \phi; \mathbf{X}) \simeq \tilde{\mathcal{L}}^M(\theta, \phi; \mathbf{X}^M) = \frac{N}{M} \sum_{i=1}^M \tilde{\mathcal{L}}(\theta, \phi; \mathbf{x}^{(i)}) \quad (8)$$

where the minibatch $\mathbf{X}^M = \{\mathbf{x}^{(i)}\}_{i=1}^M$ is a randomly drawn sample of M datapoints from the full dataset $\mathbf{X}$ with N datapoints. In our experiments we found that the number of samples L per datapoint can be set to 1 as long as the minibatch size M was large enough, e.g. $M = 100$. Derivatives $\nabla_{\theta, \phi} \tilde{\mathcal{L}}(\theta; \mathbf{X}^M)$ can be taken, and the resulting gradients can be used in conjunction with stochastic optimization methods such as SGD or Adagrad [DHS10]. See algorithm 1 for a basic approach to compute the stochastic gradients.

A connection with auto-encoders becomes clear when looking at the objective function given at eq. (7). The first term is (the KL divergence of the approximate posterior from the prior) acts as a regularizer, while the second term is an expected negative reconstruction error. The function $g_\phi(\cdot)$ is chosen such that it maps a datapoint $\mathbf{x}^{(i)}$ and a random noise vector $\epsilon^{(i,l)}$ to a sample from the approximate posterior for that datapoint: $\mathbf{z}^{(i,l)} = g_\phi(\epsilon^{(i,l)}, \mathbf{x}^{(i)})$ where $\mathbf{z}^{(i,l)} \sim q_\phi(\mathbf{z}|\mathbf{x}^{(i)})$. Subsequently, the sample $\mathbf{z}^{(i,l)}$ is then input to function $\log p_\theta(\mathbf{x}^{(i)}|\mathbf{z}^{(i,l)})$, which equals the probability density (or mass) of datapoint $\mathbf{x}^{(i)}$ under the generative model, given $\mathbf{z}^{(i,l)}$. This term is a negative *reconstruction error* in auto-encoder parlance.

2.4 The reparameterization trick

In order to solve our problem we invoked an alternative method for generating samples from $q_\phi(\mathbf{z}|\mathbf{x})$. The essential parameterization trick is quite simple. Let $\mathbf{z}$ be a continuous random variable, and $\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})$ be some conditional distribution. It is then often possible to express the random variable $\mathbf{z}$ as a deterministic variable $\mathbf{z} = g_\phi(\epsilon, \mathbf{x})$, where ϵ is an auxiliary variable with independent marginal $p(\epsilon)$, and $g_\phi(\cdot)$ is some vector-valued function parameterized by ϕ .

This reparameterization is useful for our case since it can be used to rewrite an expectation w.r.t $q_\phi(\mathbf{z}|\mathbf{x})$ such that the Monte Carlo estimate of the expectation is differentiable w.r.t. ϕ . A proof is as follows. Given the deterministic mapping $\mathbf{z} = g_\phi(\epsilon, \mathbf{x})$ we know that $q_\phi(\mathbf{z}|\mathbf{x}) \prod_i dz_i = p(\epsilon) \prod_i d\epsilon_i$. Therefore¹, $\int q_\phi(\mathbf{z}|\mathbf{x}) f(\mathbf{z}) d\mathbf{z} = \int p(\epsilon) f(\mathbf{z}) d\epsilon = \int p(\epsilon) f(g_\phi(\epsilon, \mathbf{x})) d\epsilon$. It follows

¹Note that for infinitesimals we use the notational convention $d\mathbf{z} = \prod_i dz_i$

that a differentiable estimator can be constructed: $\int q_\phi(\mathbf{z}|\mathbf{x})f(\mathbf{z})d\mathbf{z} \simeq \frac{1}{L}\sum_{l=1}^L f(g_\phi(\mathbf{x}, \epsilon^{(l)}))$ where $\epsilon^{(l)} \sim p(\epsilon)$. In section 2.3 we applied this trick to obtain a differentiable estimator of the variational lower bound.

Take, for example, the univariate Gaussian case: let $z \sim p(z|x) = \mathcal{N}(\mu, \sigma^2)$. In this case, a valid reparameterization is $z = \mu + \sigma\epsilon$, where ϵ is an auxiliary noise variable $\epsilon \sim \mathcal{N}(0, 1)$. Therefore, $\mathbb{E}_{\mathcal{N}(z;\mu,\sigma^2)}[f(z)] = \mathbb{E}_{\mathcal{N}(\epsilon;0,1)}[f(\mu + \sigma\epsilon)] \simeq \frac{1}{L}\sum_{l=1}^L f(\mu + \sigma\epsilon^{(l)})$ where $\epsilon^{(l)} \sim \mathcal{N}(0, 1)$.

For which $q_\phi(\mathbf{z}|\mathbf{x})$ can we choose such a differentiable transformation $g_\phi(\cdot)$ and auxiliary variable $\epsilon \sim p(\epsilon)$? Three basic approaches are:

1. Tractable inverse CDF. In this case, let $\epsilon \sim \mathcal{U}(\mathbf{0}, \mathbf{I})$, and let $g_\phi(\epsilon, \mathbf{x})$ be the inverse CDF of $q_\phi(\mathbf{z}|\mathbf{x})$. Examples: Exponential, Cauchy, Logistic, Rayleigh, Pareto, Weibull, Reciprocal, Gompertz, Gumbel and Erlang distributions.
2. Analogous to the Gaussian example, for any "location-scale" family of distributions we can choose the standard distribution (with location = 0, scale = 1) as the auxiliary variable ϵ , and let $g(\cdot) = \text{location} + \text{scale} \cdot \epsilon$. Examples: Laplace, Elliptical, Student's t, Logistic, Uniform, Triangular and Gaussian distributions.
3. Composition: It is often possible to express random variables as different transformations of auxiliary variables. Examples: Log-Normal (exponentiation of normally distributed variable), Gamma (a sum over exponentially distributed variables), Dirichlet (weighted sum of Gamma variates), Beta, Chi-Squared, and F distributions.

When all three approaches fail, good approximations to the inverse CDF exist requiring computations with time complexity comparable to the PDF (see e.g. [Dev86] for some methods).

3 Example: Variational Auto-Encoder

In this section we'll give an example where we use a neural network for the probabilistic encoder $q_\phi(\mathbf{z}|\mathbf{x})$ (the approximation to the posterior of the generative model $p_\theta(\mathbf{x}, \mathbf{z})$) and where the parameters ϕ and θ are optimized jointly with the AEVB algorithm.

Let the prior over the latent variables be the centered isotropic multivariate Gaussian $p_\theta(\mathbf{z}) = \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$. Note that in this case, the prior lacks parameters. We let $p_\theta(\mathbf{x}|\mathbf{z})$ be a multivariate Gaussian (in case of real-valued data) or Bernoulli (in case of binary data) whose distribution parameters are computed from $\mathbf{z}$ with a MLP (a fully-connected neural network with a single hidden layer, see appendix C). Note the true posterior $p_\theta(\mathbf{z}|\mathbf{x})$ is in this case intractable. While there is much freedom in the form $q_\phi(\mathbf{z}|\mathbf{x})$, we'll assume the true (but intractable) posterior takes on a approximate Gaussian form with an approximately diagonal covariance. In this case, we can let the variational approximate posterior be a multivariate Gaussian with a diagonal covariance structure²:

$$\log q_\phi(\mathbf{z}|\mathbf{x}^{(i)}) = \log \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}^{(i)}, \boldsymbol{\sigma}^{2(i)}\mathbf{I}) \quad (9)$$

where the mean and s.d. of the approximate posterior, $\boldsymbol{\mu}^{(i)}$ and $\boldsymbol{\sigma}^{(i)}$, are outputs of the encoding MLP, i.e. nonlinear functions of datapoint $\mathbf{x}^{(i)}$ and the variational parameters ϕ (see appendix C).

As explained in section 2.4, we sample from the posterior $\mathbf{z}^{(i,l)} \sim q_\phi(\mathbf{z}|\mathbf{x}^{(i)})$ using $\mathbf{z}^{(i,l)} = g_\phi(\mathbf{x}^{(i)}, \epsilon^{(l)}) = \boldsymbol{\mu}^{(i)} + \boldsymbol{\sigma}^{(i)} \odot \epsilon^{(l)}$ where $\epsilon^{(l)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. With $\odot$ we signify an element-wise product. In this model both $p_\theta(\mathbf{z})$ (the prior) and $q_\phi(\mathbf{z}|\mathbf{x})$ are Gaussian; in this case, we can use the estimator of eq. (7) where the KL divergence can be computed and differentiated without estimation (see appendix B). The resulting estimator for this model and datapoint $\mathbf{x}^{(i)}$ is:

$$\mathcal{L}(\theta, \phi; \mathbf{x}^{(i)}) \simeq \frac{1}{2} \sum_{j=1}^J \left(1 + \log((\sigma_j^{(i)})^2) - (\mu_j^{(i)})^2 - (\sigma_j^{(i)})^2 \right) + \frac{1}{L} \sum_{l=1}^L \log p_\theta(\mathbf{x}^{(i)}|\mathbf{z}^{(i,l)})$$

where $\mathbf{z}^{(i,l)} = \boldsymbol{\mu}^{(i)} + \boldsymbol{\sigma}^{(i)} \odot \epsilon^{(l)}$ and $\epsilon^{(l)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ (10)

As explained above and in appendix C, the decoding term $\log p_\theta(\mathbf{x}^{(i)}|\mathbf{z}^{(i,l)})$ is a Bernoulli or Gaussian MLP, depending on the type of data we are modelling.

²Note that this is just a (simplifying) choice, and not a limitation of our method.

4 Related work

The wake-sleep algorithm [HDFN95] is, to the best of our knowledge, the only other on-line learning method in the literature that is applicable to the same general class of continuous latent variable models. Like our method, the wake-sleep algorithm employs a recognition model that approximates the true posterior. A drawback of the wake-sleep algorithm is that it requires a concurrent optimization of two objective functions, which together do not correspond to optimization of (a bound of) the marginal likelihood. An advantage of wake-sleep is that it also applies to models with discrete latent variables. Wake-Sleep has the same computational complexity as AEVB per datapoint.

Stochastic variational inference [HBWP13] has recently received increasing interest. Recently, [BJP12] introduced a control variate schemes to reduce the high variance of the naïve gradient estimator discussed in section 2.1, and applied to exponential family approximations of the posterior. In [RGB13] some general methods, i.e. a control variate scheme, were introduced for reducing the variance of the original gradient estimator. In [SK13], a similar reparameterization as in this paper was used in an efficient version of a stochastic variational inference algorithm for learning the natural parameters of exponential-family approximating distributions.

The AEVB algorithm exposes a connection between directed probabilistic models (trained with a variational objective) and auto-encoders. A connection between *linear* auto-encoders and a certain class of generative linear-Gaussian models has long been known. In [Row98] it was shown that PCA corresponds to the maximum-likelihood (ML) solution of a special case of the linear-Gaussian model with a prior $p(\mathbf{z}) = \mathcal{N}(0, \mathbf{I})$ and a conditional distribution $p(\mathbf{x}|\mathbf{z}) = \mathcal{N}(\mathbf{x}; \mathbf{W}\mathbf{z}, \epsilon\mathbf{I})$, specifically the case with infinitesimally small ϵ .

In relevant recent work on autoencoders [VLL⁺10] it was shown that the training criterion of unregularized autoencoders corresponds to maximization of a lower bound (see the infomax principle [Lin89]) of the mutual information between input X and latent representation Z . Maximizing (w.r.t. parameters) of the mutual information is equivalent to maximizing the conditional entropy, which is lower bounded by the expected loglikelihood of the data under the autoencoding model [VLL⁺10], i.e. the negative reconstruction error. However, it is well known that this reconstruction criterion is in itself not sufficient for learning useful representations [BCV13]. Regularization techniques have been proposed to make autoencoders learn useful representations, such as denoising, contractive and sparse autoencoder variants [BCV13]. The SGVB objective contains a regularization term dictated by the variational bound (e.g. eq. (10)), lacking the usual nuisance regularization hyperparameter required to learn useful representations. Related are also encoder-decoder architectures such as the predictive sparse decomposition (PSD) [KRL08], from which we drew some inspiration. Also relevant are the recently introduced Generative Stochastic Networks [BTL13] where noisy auto-encoders learn the transition operator of a Markov chain that samples from the data distribution. In [SL10] a recognition model was employed for efficient learning with Deep Boltzmann Machines. These methods are targeted at either unnormalized models (i.e. undirected models like Boltzmann machines) or limited to sparse coding models, in contrast to our proposed algorithm for learning a general class of directed probabilistic models.

The recently proposed DARN method [GMW13], also learns a directed probabilistic model using an auto-encoding structure, however their method applies to binary latent variables. Even more recently, [RMW14] also make the connection between auto-encoders, directed probabilistic models and stochastic variational inference using the reparameterization trick we describe in this paper. Their work was developed independently of ours and provides an additional perspective on AEVB.

5 Experiments

We trained generative models of images from the MNIST and Frey Face datasets³ and compared learning algorithms in terms of the variational lower bound, and the estimated marginal likelihood.

The generative model (encoder) and variational approximation (decoder) from section 3 were used, where the described encoder and decoder have an equal number of hidden units. Since the Frey Face data are continuous, we used a decoder with Gaussian outputs, identical to the encoder, except that the means were constrained to the interval $(0, 1)$ using a sigmoidal activation function at the

³Available at <http://www.cs.nyu.edu/~roweis/data.html>